

// Jivops

NATS

Guía completa — 20 lecciones

Mensajería central · JetStream · Clustering y topologías · Seguridad y operación

Contenido

Módulo 1: Fundamentos y mensajería central

- Lección 01 — Qué es NATS y su modelo de mensajería
- Lección 02 — Subjects y wildcards
- Lección 03 — Publicación/suscripción y entrega como máximo una vez
- Lección 04 — Queue groups
- Lección 05 — Request-Reply

Módulo 2: JetStream

- Lección 06 — JetStream: persistencia sobre NATS
- Lección 07 — Streams y políticas de retención
- Lección 08 — Consumers: durables, efímeros, push y pull
- Lección 09 — Acks, reentregas y deduplicación
- Lección 10 — Key-Value y Object Store

Módulo 3: Clustering y topologías

- Lección 11 — Clustering de servidores NATS
- Lección 12 — Gateways y superclusters
- Lección 13 — Leaf nodes
- Lección 14 — JetStream en cluster: réplicas y Raft
- Lección 15 — Mirrors y sources

Módulo 4: Seguridad y operación

- Lección 16 — Autenticación: tokens, usuarios y NKeys
- Lección 17 — Cuentas y multi-tenancy
- Lección 18 — JWT descentralizado y TLS
- Lección 19 — Monitorización y observabilidad
- Lección 20 — Proyecto final: NATS listo para producción

Módulo 1: Fundamentos y mensajería central

Lección 01 — Qué es NATS y su modelo de mensajería

Un sistema de mensajería ligero y de alto rendimiento, escrito en Go, donde los clientes se conectan al servidor nats-server (puerto 4222 por defecto) y se comunican por subjects; el núcleo (Core NATS) no persiste mensajes y la persistencia se añade con JetStream.

Core NATS no guarda mensajes en disco y su entrega es como máximo una vez, lo que reduce la latencia; si un suscriptor no está conectado, pierde el mensaje.

Lección 02 — Subjects y wildcards

Un subject es una cadena de tokens separados por puntos; el wildcard * sustituye exactamente un token y el wildcard > sustituye uno o más tokens y solo puede ir al final, por lo que pedidos.* casa con pedidos.nuevo y pedidos.> casa también con pedidos.eu.nuevo.

* casa con un único token (pedidos.* no casa con pedidos.eu.nuevo) y > casa con uno o más tokens y solo puede ir al final del subject.

Lección 03 — Publicación/suscripción y entrega como máximo una vez

Entrega como máximo una vez (at-most-once): el servidor reenvía cada mensaje a los suscriptores conectados en ese momento, sin persistirlo ni reintentarlo, por lo que un suscriptor desconectado no lo recibe después.

No recibirá ninguno: Core NATS solo entrega a suscriptores conectados y no guarda mensajes; para recuperarlos haría falta JetStream.

Lección 04 — Queue groups

Reparte los mensajes de un subject entre los suscriptores que comparten el mismo nombre de grupo, de modo que cada mensaje lo procesa solo uno de los miembros, lo que permite balancear carga y escalar consumidores sin cambios en el publicador.

Lanzas más suscriptores con el mismo nombre de queue group; el servidor reparte cada mensaje a uno solo de ellos y el publicador no cambia.

Lección 05 — Request-Reply

El solicitante publica con un subject de respuesta único (inbox, normalmente _INBOX.<id>) en el que escucha, y el servicio contesta publicando en ese subject; si no hay ningún suscriptor para el subject de la petición, el servidor puede devolver un aviso de «no responders» sin esperar al timeout.

Si nadie escucha el subject, el cliente se entera al instante y falla rápido, sin esperar al timeout completo.

Módulo 2: JetStream

Lección 06 — JetStream: persistencia sobre NATS

Persistencia y reproducción de mensajes sobre Core NATS: guarda los mensajes en streams, permite consumirlos más tarde con confirmaciones (entrega al menos una vez) y se activa en el propio nats-server, sin añadir un componente externo.

Core NATS pierde los mensajes si no hay suscriptor; JetStream los guarda en streams, permite consumirlos después y ofrece entrega al menos una vez con acks.

Lección 07 — Streams y políticas de retención

Un stream almacena los mensajes publicados en uno o varios subjects, en disco (file) o memoria, y su retención puede ser por límites (limits), por interés (interest) o como cola de trabajo (workqueue), donde cada mensaje se elimina al ser confirmado.

Cuando el stream actúa como cola de trabajo: cada mensaje debe procesarse una sola vez y se elimina al ser confirmado, en vez de conservarse hasta un límite.

Lección 08 — Consumers: durables, efímeros, push y pull

Una vista con estado sobre un stream que recuerda qué mensajes se han entregado y confirmado; puede ser durable (se conserva con nombre) o efímero (se elimina sin actividad), y de tipo push (el servidor envía) o pull (el cliente solicita lotes).

El durable tiene nombre y conserva su posición aunque se desconecte; el efímero se elimina tras un periodo de inactividad y pierde su estado.

Lección 09 — Acks, reentregas y deduplicación

Con la política de ack explícita, el consumer debe confirmar cada mensaje; si no llega el ack antes de AckWait, el servidor lo reentrega hasta MaxDeliver veces, y el cliente puede responder Nak para reintentar o Term para no reentregar más.

Configurando MaxDeliver para limitar los reintentos y, si la aplicación detecta que es irrecuperable, respondiendo Term para que no se reentregue más.

Lección 10 — Key-Value y Object Store

Abstracciones construidas sobre streams de JetStream: el KV guarda el último valor de cada clave con historial y permite observar cambios (watch), y el Object Store guarda objetos grandes troceándolos en mensajes.

Porque cada bucket KV es en realidad un stream de JetStream, así que hereda persistencia, réplicas y reentrega.

Módulo 3: Clustering y topologías

Lección 11 — Clustering de servidores NATS

Los servidores se conectan entre sí por rutas (puerto de cluster 6222 por convención) formando una malla completa (full mesh); basta indicar algunas rutas semilla y los demás servidores se descubren por gossip, y los clientes pueden reconectarse a cualquier nodo.

Si un nodo cae, el cliente se reconecta automáticamente a otro; además descubre los servidores del cluster, lo que da alta disponibilidad.

Lección 12 — Gateways y superclusters

Para unir varios clusters en un supercluster, normalmente en regiones distintas, con conexiones optimizadas entre ellos: cada gateway solo reenvía a otro cluster los mensajes con interés en ese subject, y el tráfico prefiere el cluster más cercano.

Cada región mantiene su cluster local y los gateways solo reenvían los mensajes con interés remoto, evitando una malla completa con alta latencia.

Lección 13 — Leaf nodes

Un servidor NATS que se conecta como cliente a otro servidor o cluster para extender el sistema, manteniendo su propia autenticación y sus propias cuentas, útil en el borde (edge), en redes con firewall o para aislar entornos.

En una sede o dispositivo de borde: el leaf funciona localmente aunque se corte la conexión, abre solo una conexión saliente y aplica su propia autenticación.

Lección 14 — JetStream en cluster: réplicas y Raft

Cada stream tiene un número de réplicas (R1, R3, R5) y usa el consenso Raft: una escritura se confirma cuando la mayoría de las réplicas la ha aceptado, por lo que R3 tolera la caída de un nodo y se recomienda un número impar.

Uno: Raft necesita la mayoría (2 de 3) para confirmar escrituras y elegir líder; con dos nodos caídos se pierde el quórum.

Lección 15 — Mirrors y sources

Un mirror es una copia de solo lectura de un único stream (sigue su contenido), mientras que un stream con sources agrega mensajes de uno o varios streams y también puede recibir publicaciones propias; ambos sirven para replicar entre clusters.

Sources: permite agregar mensajes de varios streams en uno; un mirror solo puede copiar un único stream.

Módulo 4: Seguridad y operación

Lección 16 — Autenticación: tokens, usuarios y NKeys

Token único, usuario y contraseña (con contraseñas en bcrypt), certificados TLS de cliente y NKeys, pares de claves Ed25519 donde el servidor solo conoce la clave pública y el cliente firma un reto con su clave privada, que nunca se envía.

Porque el cliente demuestra su identidad firmando un reto con su clave privada, que nunca viaja por la red; un token compartido sí puede interceptarse o filtrarse.

Lección 17 — Cuentas y multi-tenancy

Espacios aislados de subjects: los mensajes de una cuenta no son visibles para otra salvo que se exporten e importen servicios o streams de forma explícita, lo que permite multi-tenancy en un mismo servidor o cluster.

Una cuenta exporta un servicio o stream y la otra lo importa de forma explícita; fuera de eso, los subjects siguen siendo invisibles entre cuentas.

Lección 18 — JWT descentralizado y TLS

Una jerarquía de confianza operator → cuentas → usuarios firmada con NKeys, donde el servidor solo necesita la clave del operator y se pueden crear o revocar cuentas y usuarios sin editar ni reiniciar la configuración de cada servidor; se gestiona con la herramienta nsc.

Permite crear y revocar cuentas y usuarios sin editar ni reiniciar los servidores, con una gestión descentralizada basada en firmas.

Lección 19 — Monitorización y observabilidad

Con el puerto de monitorización (8222 por convención), que expone endpoints HTTP como /varz, /connz, /routez y /jsz con métricas del servidor, conexiones, rutas y JetStream, y se integra con Prometheus mediante un exporter.

Es un slow consumer: el cliente no lee al ritmo que llegan los mensajes y su buffer se llena; se detecta en los logs y en las métricas de /varz y /connz.

Lección 20 — Proyecto final: NATS listo para producción

Cluster de tres o más nodos con streams R3, TLS y autenticación (NKeys o JWT con cuentas), límites de tamaño y de conexiones, monitorización con Prometheus y alertas, y copias de seguridad o mirrors de los streams críticos.

Streams con réplicas R3 sobre un cluster impar, TLS y autenticación con cuentas, y monitorización con alertas (más límites y copias de los streams críticos).